DATABASE, AI DEBUGGER & INTEGRATION

Individual Work Plan • Person 4 • Your Responsibilities

1. Mission

You own the persistence and AI-debugging layer, and coordinate the final integration of the four modules. Your goal is to store workspace and file data reliably, provide APIs that the frontend can call, connect the AI debugger to relevant code and execution errors, and verify that the complete IDE works end to end.

Core outcome: users can save and reopen workspace files, ask the AI to explain an error and suggest a fix, and use the integrated app without the four modules breaking at their boundaries.

2. Workstream A — PostgreSQL and Persistence

- Choose/configure the PostgreSQL instance and store connection settings in server-side environment variables.
- Create tables for users, rooms/workspaces, and workspace_files, with primary keys, foreign keys, timestamps, and a uniqueness rule for file paths within a workspace.
- Implement APIs to create and retrieve workspaces and save/load file content.
- Define the save strategy with Adarsh: live Yjs state is not automatically durable just because it is synchronized over WebSockets.
- Decide when file contents are persisted (for example, debounced saves or explicit save events) and how data is restored after a server restart.
- Validate request data and return consistent success/error responses.

3. Workstream B — AI Debugger

- Implement a server-side endpoint such as POST /api/ai/debug (agree on the final path with Jhostan).
- Accept the relevant source code, language, error message, and stack trace when available.
- Call the OpenAI API from the backend; never place the API key in browser code.
- Return a clear explanation, likely cause, and suggested fix in a predictable response format.
- Display or pass the response to Jhostan's frontend panel.
- Require the user to review and confirm a proposed fix before applying it to the shared Yjs document.
- Handle missing context, API errors, timeouts, and rate limits gracefully.

4. Workstream C — Integration Coordinator

- Agree on API contracts and data formats with all team members before integration.
- Track the interface boundaries: Jhostan owns the UI; Adarsh owns Yjs/WebSocket collaboration; Ashish owns browser execution and terminal service; you own persistence and AI.
- Provide a single .env.example listing variable names without real secret values.
- Check CORS, ports, API base URLs, WebSocket URLs, and local/deployed configuration.
- Maintain a short integration checklist and test the application in a clean environment.
- Coordinate merges and resolve integration issues; avoid making unreviewed changes to teammates' modules.

5. Suggested Database Schema

```
users: id, username, avatar_url, created_at
rooms: id, name, owner_id, created_at
workspace_files: id, room_id, path, content, updated_at
Constraint: UNIQUE(room_id, path)
```

This follows the relational entities in the supplied architecture blueprint. Add authentication-related fields or other tables only if the team chooses to implement those features.

6. Integration Contracts to Confirm

Module	Contract to agree	Owner
Workspace API	Create/list/get workspace; stable workspace ID; error format.	You + Jhostan
File API	Save/load path and content; response format; save timing.	You + Jhostan
Yjs persistence	How collaborative document state is saved and restored.	You + Adarsh
AI debug API	Request includes code/language/error; response contains explanation and suggested fix.	You + Jhostan
Execution errors	How Ashish's runner returns stdout/stderr/status for AI context.	You + Ashish

7. Suggested 24-Hour Roadmap

Time	Tasks	Milestone
Hours 0–3	Agree on API/data contracts; set up PostgreSQL and environment variables.	Schema ready.
Hours 3–7	Implement workspace and file save/load APIs.	Files persist.
Hours 7–10	Agree on Yjs persistence strategy and test restore behavior.	Persistence plan verified.
Hours 10–14	Implement server-side AI debug endpoint and response shape.	AI endpoint works.
Hours 14–18	Connect frontend debugger; pass execution errors into the request.	AI flow integrated.
Hours 18–21	Run full-system tests with all three teammates; fix contract/configuration issues.	Core flows pass.
Hours 21–24	Test clean startup, persistence after restart, secrets/configuration, and demo flow.	Demo-ready app.

8. Acceptance Checklist

- ■ PostgreSQL schema initializes using documented instructions.
- ■ Workspace and file APIs can save and retrieve content.
- ■ Saved file content survives backend restart.
- ■ Collaborative document persistence behavior is agreed with Adarsh and tested.
- ■ AI endpoint accepts code and relevant error context and returns a useful structured response.
- ■ OpenAI API key remains server-side and is not committed to source control.
- ■ AI suggestions are reviewed before being applied to shared code.
- ■ Frontend, collaboration, execution, database, and AI components work together in one demo.
- ■ A clean setup guide and .env.example are available.
- ■ Known limitations and any unfinished features are documented honestly.

9. Security and Scope Notes

- • Never commit .env files, database passwords, or API keys.
- • Validate and limit code/error payload sizes sent to the AI endpoint.
- • Do not automatically execute AI-generated code or apply fixes without user confirmation.
- • Apply authorization to workspace/file endpoints before public deployment.
- • Keep the MVP focused: reliable save/load and one useful AI debugging workflow are better than several unfinished features.

Definition of done: workspace files can be saved and restored, the AI debugger can explain an execution error and suggest a fix, and the complete four-person application passes a shared end-to-end demo.